

02. Python Fundamentals (Part-2)

Complete Learning Guide: Conditional Statements, Match Case, Loops & Functions

Author: Golam Maula Lincoln

GitHub Profile: <https://github.com/gmlincoln>

Visit GitHub Profile [↗](https://github.com/gmlincoln)

1. Conditional Statements

Conditional statements let our program decide what to do based on conditions (i.e. True or False expressions).

There are 3 main conditional statements in Python:

1. **if** - used to test a condition. If it's True, the indented block runs.
2. **else** - the else block runs if all above conditions are false.
3. **elif** - ("else if") used when we have multiple conditions to check.

Code Examples

Example 1: Age Verification

```
age = int(input("enter age: "))

if age >= 18:
    # if true/false, entire block of code is executed
    print("you can vote")
    print("you can drive")
else:
    print("you can't vote")
    print("you can't drive")
```

Example 2: Traffic Light System

```
color = input("enter color: ")

if color == "red":
    print("Stop")
elif color == "yellow":
    print("Look")
elif color == "green":
    print("Go")
else:
    print("wrong color")
```

Note: We can have standalone **if** statements, but **elif** and **else** cannot be used without an **if** statement preceding them.

Logical Operators in Expressions & Nesting

Apart from simple conditions, we can also check for multiple conditions using logical operators like **and**, **or**, and **not** in expressions.

```
# Example 3 - Logical operators in expressions
age = int(input("enter age: "))

if age < 13:
    print("child")
elif (age >= 13 and age < 18):
    print("teenager")
else:
    print("adult")
```

Nesting in Conditionals: Nesting means placing one block of code inside another. In conditional statements, it means putting one if-else (or if-elif-else) inside another.

```
# Login System with Nesting
username = input("enter username: ")
password = input("enter password: ")

if (username == "admin" and password == "pass"):
    print("log in successful!")
else:
    if username != "admin": # NESTING
        print("wrong user name, try again.")
    else:
        print("wrong password, try again.")
```

Ternary Statements (Conditional Expressions)

Ternary statements are a compact way to write if-else statements in one line.

```
value_if_true if condition else value_if_false

# Example
age = 18
status = "Adult" if age >= 18 else "Not Adult"
print(status)
```

Note: Everything written with ternary statements can be done with normal if-else statements. Ternary statements are recommended only for simple conditions.

2. Match Case Statements

In Python (introduced in Python 3.10), match/case statements provide an alternative to long chains of if-elif-else statements.

Syntax

```
match variable:
    case pattern1:
        # code to run if pattern1 matches
    case pattern2:
        # code to run if pattern2 matches
    case _:
        # default case (wildcard, like 'else')
```

- match - what you are comparing
- case - the value or pattern to match against

- `_` - wildcard (matches anything, acts as default)

Example: Traffic Lights using Match Case

```
color = input("enter color: ")

match color:
    case "red":
        print("Stop")
    case "yellow":
        print("Look")
    case "green":
        print("Go")
    case _:
        print("wrong color")
```

3. Loops

A loop lets us execute a block of code multiple times: either a set number of times or until a condition is met. Python has 2 main types of loops: while loops and for loops.

A. The while Loop

```
while condition:
    # code block
```

```
# Example 1 - Print 1 to 5
i = 1
while i <= 5:
    print(i)
    i += 1

# Example 2 - Print 5 to 1
i = 5
while i > 0:
    print(i)
    i -= 1
```

Infinite Loop Warning: If a loop expression always evaluates to `True`, the loop will run forever. Avoid creating unintended infinite loops (e.g., `while True: print("Prime")`).

B. Loop Control Statements: `break` & `continue`

- `break` - Stops the loop execution immediately.
- `continue` - Skips the rest of the current iteration and moves to the next one.

```
# Break Example - Stop at multiple of 6
i = 1
while i <= 10:
    if (i % 6 == 0):
        break
    print(i)
    i += 1
# Output: 1, 2, 3, 4, 5

# Continue Example - Skip multiples of 3
i = 0
while (i < 10):
    i += 1
    if (i % 3 == 0):
        continue
    print(i)
# Output: 1, 2, 4, 5, 7, 8, 10
```

C. The for Loop & Membership Operator

A for loop iterates over a sequence (such as a list, string, or range) and executes code for each item.

```
for variable in sequence:
    # code to run for each item
```

```
# Looping over a string
word = "Prime"
for ch in word:
    print(ch)

# Check membership
if 'i' in word:
    print("letter exists")

# Count occurrences
word = "artificial intelligence"
count = 0
for ch in word:
    if ch == 'i':
        count += 1
print(f"i occurs {count} times.")
```

Nested Loops: Loop inside a loop.

```
for i in range(1, 3):
    for j in range(1, 3):
        print(f"({i}, {j})")
# Output: (1, 1), (1, 2), (2, 1), (2, 2)
```

D. The range() Function

Generates a sequence of numbers efficiently on demand. Parameters: range(start, stop, step).

```
# 1 Argument (stop)
for i in range(5): print(i) # Output: 0, 1, 2, 3, 4

# 2 Arguments (start, stop)
for i in range(1, 6): print(i) # Output: 1, 2, 3, 4, 5

# 3 Arguments (start, stop, step)
for i in range(1, 10, 2): print(i) # Output: 1, 3, 5, 7, 9
```

4. Functions

Functions are blocks of reusable code defined using the `def` keyword.

```
def hello():  
    print("hello from Prime")  
  
hello() # Function Call
```

Parameters, Arguments & Return Values

```
# Function with parameters  
def sum(a, b):  
    print(a + b)  
  
sum(5, 10) # Arguments 5 and 10  
  
# Function returning a value  
def avg(a, b, c):  
    return (a + b + c) / 3  
  
print(avg(1, 2, 3))
```

Default Parameters

Default parameters must come last after non-default parameters.

```
def sum(a, b=1):  
    return a + b  
  
print(sum(5)) # Output: 6
```

Built-in Functions vs Lambda Functions

Built-in functions like `print()`, `input()`, `type()`, and `range()` come pre-defined in Python.

Lambda Functions: One-liner anonymous functions created with the `lambda` keyword.

```
syntax: lambda arguments: expression  
  
# Lambda to compute square  
square = lambda x: x * x  
print(square(5)) # Output: 25
```

5. Complete Solutions to Practice Problems

Set 1: Conditionals Solutions

```
# 1. Multiple of 5 or not
num = int(input("enter number: "))
if num % 5 == 0:
    print("multiple of 5")
else:
    print("NOT a multiple of 5")

# 2. Odd or Even
num = int(input("enter number: "))
if num % 2 == 0:
    print("Even")
else:
    print("Odd")
```

Set 2: Loops Solutions

```
# 1. Multiplication table of n (using while loop)
n = int(input("enter n: "))
i = 1
while i <= 10:
    print(i * n)
    i += 1

# 2. Odd numbers from 1 to 10 (using continue)
i = 0
while i < 10:
    i += 1
    if i % 2 == 0:
        continue
    print(i) # Output: 1, 3, 5, 7, 9

# 3. Count vowels in a word (using for loop)
word = input("enter word: ")
count = 0
for ch in word.lower():
    if ch in ['a', 'e', 'i', 'o', 'u']:
        count += 1
print(f"vowel count = {count}")

# 4. Sum of first n Natural numbers (using range)
n = int(input("enter n: "))
total = 0
for i in range(1, n + 1):
    total += i
print("sum =", total)
```

Set 3: Functions Solutions

```
# 1. Factorial of N
n = int(input("enter n: "))
fact = 1
for i in range(1, n + 1):
    fact *= i
print("factorial =", fact)

# 2. Largest of 3 numbers
def get_largest(a, b, c):
    if (a > b and a > c):
        return a
    elif b > c:
        return b
    else:
        return c

print(get_largest(3, 10, 5)) # Output: 10
```

✨ Keep Learning & Keep Exploring! ✨

Document created for GitHub profile: [gmlincoln \(Golam Maula Lincoln\)](https://github.com/gmlincoln)